

Xesta Documentation

Contents

- Xesta..... 3**
 - An adjusted Java Object / Xml Document Binding.....3
 - Context..... 3

- Getting Started with Xesta..... 3**
 - Download and configure the Fento jar and dependencies.....3
 - Annotate your first class.....3
 - Retrieve the document.....4

- Repository types.....4**
 - Path Adjustment.....4
 - File/Folder based repository.....4
 - Classpath repository.....4
 - REST based repository.....4
 - Zip file based repository.....5
 - Abstract repository.....5

- Bind Annotation Reference..... 5**

- Xesta..... 6**
 - Status..... 6

- Changelog.....6**

- Acknowledgements.....6**

- References..... 6**

Xesta

An adjusted Java Object / Xml Document Binding

Xesta is a java to xml binding tool. Xesta balances the approach between object and document side of the binding, trying to rely on tooling and practices aligned with each of the sides.

Xesta was presented as Fento on XML Prague 2026 and afterwards renamed as Xesta.

Context.

Document (Xml) information is usually more complex, updated more frequently, including the data structure and may be more focused on small details. Documents, Xml documents, are a great way of to handle detail and persistent, isolated (not directly backed by any system) data. They are also the backbone of the publishing and digital humanities industry, including legal texts, books, product instructions, etc.

Xml Documents are usually explored with XPath indications. While Java comes by default with XPath 1.x the Xml community rely on newer versions. Xesta annotations are defined with XPath 3.x instead of positional indications. Future XPath versions may be adopted. Xesta relies heavily on Saxon, the mighty product from Saxonica to perform the XPath queries.

On the other side, Java and object oriented tools tend to work with more structured data, less complex but with more volume. This part require more consistency and a more structured/less flexible approach. Java tooling frequently rely on Xml schemas when dealing with Xml documents. It is also common for Java tools to look for similar structures on both sides and rely on Xml for plain object serialization.

Getting Started with Xesta.

Download and configure the Fento jar and dependencies.

Download the jar and dependencies or declare the dependency on the maven pom file.

Options:

- Download the last version from <http://github.com/vionta/xesta>
- Declare the dependency on the pom file.

Annotate your first class.

Xesta defines the xml document locations using Java annotations (Bind), the Xml locaitons are defined with the "expression" attribute. Classes should implement the Serializable interface.

```
@Bind(expression = "/*:CstmrCdtTrfIntrn" ) public class CheckNoNS implements Serializable {
```

They should also have a no args constructor.

```
public CheckNoNS() {}
```

Then you should annotate the fields.

```
@Bind(expression = ":PmtInf/:DbtrAcct/:Id/:Othr/*:Id") String debtorAccount;
```

Retrieve the document

In order to retrieve the document you use a repository. The simplest repository is a File repository where you pass the file location.

```
FilePathRepository repo = new FilePathRepository(CommonTest.getBasePath()+"cases/iso20022/
GlobalPayment.xml"); GlobalPayment globalPayment = new GlobalPayment(); globalPayment =
repo.load(globalPayment);
```

You can find complete examples at <http://www.github.com/vionta/xesta-samples>.

Repository types.

Xesta access to documents are wrapped with the repository abstraction. In order to retrieve or update a document you need to choose from one of the repository types, provide the configuration parameters, usually the path or paths.

Once the repository is configured (its path or paths are provided) you request the repository to retrieve or update the document for you.

Path Adjustment.

The repository paths (in the supported cases) can be adjusted based on the object properties. In order to configure the adjustment the property name (separated with dots between objects) must be defined inside the key { } markers.

File/Folder based repository.

The simplest possible repository with a base path and a file path. The base path is not adjusted based on the the object properties, while the file path can be adjusted with the {} keys.

```
FilePathRepository repo = new FilePathRepository(CommonTest.getBasePath()+"cases/iso20022/
GlobalPayment.xml"); GlobalPayment globalPayment = new GlobalPayment(); globalPayment =
repo.load(globalPayment);
```

Classpath repository.

A repository that retrieves files from the application classpath. This repository does not allow to create new files, also some files may be available for reading but not accesible for writing. The file path is defined with simple file path conventions separating folders with the slash "/" operator.

```
ClassPathRepository repo = new ClassPathRepository().setPath("log4j.xml"); LogLevelSimple log
= repo.load(new LogLevelSimple()); // ... execute some modifications over the POJO values here...
repo.persist(log);
```

REST based repository.

A repository that retrieves and stores the files from a REST service. The file path can also be adjusted via the object data using the {} keys on the path.

**** Note: service authentication not yet developed. ****

Zip file based repository.

The zip file repository is similar to the file repository but acts on files that are placed inside zip files. Both the zipfile and the content file can be adjusted via the object properties. Zip file names are not required to end in ".zip". Zip repository can be used for odt, ods, wars, etc.

```
String zipFilePattern = CommonTest.getBasePath()+"openDocument/text/fictional-report.odt";
ZipArchiveFileRepository repo = new ZipArchiveFileRepository( zipFilePattern,"content.xml");

Document1 doc1 = new Document1(); doc1 = (Document1) repo.load(doc1);
```

Abstract repository.

An abstract implementation that supports extensibility in other cases. Still not available.

**** Note: Functionality in development. ****

Bind Annotation Reference.

The Bind annotation reference has the fo

```
import net.vionta.xml.xesta.bind.annotation.Bind;
```

- expression The XPath mapping expression that points to the selected document nodes.
- key Indicates if the attribute identifies the node element. It is only considered when the collection strategy is equal to SerializingMode. BIND_COLLECTION_BY_KEY.
- classNames [String | Default : null | Mandatory : false] The class names of a multiple collection. An element list can contain more than one single element type. Xesta looks for the mappings in all of the classes to determine the collection elements. *This feature is under development*
- serializingMode
- deserializingMode
- collectionBindStrategy
- collectionDeleteUnmatched
- namespaceAlias [String | Default : null | Mandatory : false] A list of the name space alias.
- namespaceUri [String | Default : null | Mandatory : false] A list of the name space uris.
- auto If true the element name or expression is calculated from the java property name.
- DeserializingMode
 - FAIL_ON_NOT_EXISTING : 1
 - WARN_ON_NOT_EXISTING : 2
 - AVOID_ON_NOT_EXISTING : 3
- SerializingMode
 - FAIL_ON_NOT_EXISTING : 1
 - CREATE_ON_NOT_EXISTING : 2
 - SKIP_ON_NOT_EXISTING : 3
 - BIND_COLLECTION_BY_KEY : 1
 - BIND_COLLECTION_BY_POSITION : 2
 - BIND_COLLECTION_APPEND_LAST : 3
 - BIND_COLLECTION_APPEND_FIRST : 4
 - BIND_COLLECTION_FULL_RESET : 5
 - COLLECTION_DONT_DELETE_UNMATCHED : 1
 - COLLECTION_DELETE_UNMATCHED : 2

Xesta

Status

Xesta has been developed as an experiment and a proposal for XML Prague 2026. First scheduled presentation will take place at the Prague Economics University on June 6. At this stage, Xesta could be considered as an alpha version. Anyhow, remarkable number of tests has been performed and executed.

Changelog

- 2026-06-08 Fento is renamed as Xesta. Due to a possible coincidence with another component the Fento Xml Binding component has been renamed as Xesta.

Acknowledgements.

- Saxonica The XPath heavy lifting is performed by the mighty tool. www.saxonica.com

References.

- XML Prague presentation The tool, named Fento at the time, was presented at Prague Xml in 2026. <https://www.xmlprague.cz/day3-2026/>